

qis: reproducible performance analytics and portfolio backtesting in Python

Artur Sepp¹

1 LGT Bank (Schweiz) AG, Zürich, Switzerland

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Daniel S. Katz](#)

Reviewers:

- [@alex09x](#)
- [@robertmartin8](#)
- [@tschm](#)

Submitted: 29 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

qis computes the quantities a quantitative portfolio manager or investor reports. It turns a series of portfolio weights, generated externally, and a panel of underlying asset prices into a simulated portfolio, measures it against a benchmark, and renders the result as a factsheet. In addition to the backtesting engine, qis provides strategy and reporting building blocks: exponentially weighted covariance, block bootstrap resampling, risk measures, regime-conditional statistics, currency hedging, and return unsmoothing.

We designed qis around a separation of responsibilities: the researcher supplies target weights, while the library reconstructs holdings and applies explicit accounting and reporting conventions. Signal generation and portfolio construction remain with the caller.

qis is the base layer of a stack of quantitative finance packages — optimalportfolios, trendfollowing and privateassets, all three public. Each capability is written once and reused across that stack. It requires only the scientific Python stack, runs on Python 3.10 to 3.14, and is released under the MIT licence.

Statement of need

Reported portfolio performance numbers depend on conventions that are rarely stated. Volatility depends on the return frequency, a Sharpe ratio on whether a risk-free rate is subtracted and on which rate, and a backtested return on whether holdings drift between rebalancings or are reset every period. Two implementations of the same strategy can therefore report different numbers without either containing an error.

We answer each of these questions once, inside the library, and report the answer beside the number. The return convention is an argument rather than an assumption, three Sharpe conventions are named and selected explicitly, and the reporting frequency appears on every rendered panel. The simulation holds units between rebalancings, so the realised weights drift with prices, which is what a portfolio does.

The stationary bootstrap illustrates why conventions need verification. Before version 5.1.0, qis truncated blocks at the sample boundary. Since 26 July 2026 it uses circular wrapping, as in Politis & Romano (1994). In the package's fixed synthetic example, truncation draws the first observation of a 250-period sample at 0.11 times its uniform weight and the first decile at 0.53 times. With drift rising through the sample, the resampled mean exceeds the source mean by 2.15% per year; circular wrapping reproduces the source mean within 0.32%. These are sample-specific results, reproduced by the example and pinned by regression tests. Results generated under the earlier convention therefore require the earlier software version to reproduce.

A backtest needs two inputs: a strategy generator produces weights, and a price panel aligned

to those weights supplies everything else. When each consumer of a result rebuilds the reported quantities for itself, the Sharpe ratio of one strategy, read from a chart, a summary table and a factsheet, comes out as three numbers. We build all three from one PortfolioData object constructed from weights and prices, and from the same object come the comparisons a pipeline actually runs. In addition, we implement two other base pipelines for strategy profiling: a bumped strategy against its base, and several building blocks of the same strategy side by side. All three reports are rendered by one reporting layer rather than a figure written per comparison.

State of the field

The following comparison covers selected documented workflows, rather than an exhaustive feature or performance benchmark. The references identify the software and its documentation.

package	its documented interface takes	and returns
pyfolio-reloaded (pyfolio-reloaded contributors, 2026)	returns, optionally positions and transactions	tear sheets and round-trip analysis
bt (bt contributors, 2026)	target weights through WeighTarget in an Algo stack	statistics, weights, positions and turnover
vectorbt (vectorbt contributors, 2026)	orders, signals or an order-generating callback	order and trade records, statistics and plots
Riskfolio-Lib (Riskfolio-Lib contributors, 2026)	returns and optimisation constraints	weights, reports and plots
skfolio (skfolio contributors, 2026)	returns fitted through a scikit-learn interface	weights, cross-validation, risk measures and plots
quantstats (QuantStats contributors, 2026)	returns	metrics, plots and HTML
empyrical-reloaded (empyrical-reloaded contributors, 2026)	returns	tear sheets return and risk metrics
ffn (ffn contributors, 2026)	prices	analytics; its documentation refers simulation to bt
PyPortfolioOpt (Martin, 2021)	prices, or expected returns and a risk model	weights, discrete allocation, performance and plots

arch provides a related resampling primitive: StationaryBootstrap takes several aligned arrays, Series or DataFrames in one object, so one index draw propagates across them ([arch contributors, 2026](#)).

qis differs in where the boundary sits. Only weights and prices cross it, so any generator producing weights can be measured. The simulation returns a PortfolioData object carrying the net asset value, the realised weights, the held units, the instrument-level profit and loss and the realised costs together, so attribution is a property of that object rather than a later calculation on a series that has already lost what it needs.

Building a separate library, rather than extending bt or ffn, served this shared result-object boundary across the author's research packages. bt already accepts target weights, and ffn already supplies analytics; qis combines holdings, costs, attribution and comparative reports without requiring callers to adopt an Algo stack or reconstruct those fields from returns. This choice trades the flexibility of a general strategy framework for a common accounting contract; it does not imply that the alternatives cannot be extended.

65 The integrated analytics include return unsmoothing for appraisal-based valuations (Getmansky
66 et al., 2004), regime-conditional reporting by benchmark return quantile (Sepp & Dezaud,
67 2019), and paired bootstrap resampling of aligned factor and residual panels. These methods
68 are not claimed as unique: R's PerformanceAnalytics implements Geltner unsmoothing
69 (Peterson & Carl, 2026), and arch supplies the paired resampling primitive above.

70 Software design

71 We organise the package into 14 capability groups, which `src/qis/api.py` names.

72 The backtester is 441 lines. It converts target weights to units at each rebalancing, holds those
73 units until the next one, applies costs, and returns the result. Holding units preserves natural
74 weight drift; multiplying every period's returns by unchanged target weights would instead
75 assume repeated rebalancing. This path dependence motivates a date-sequential numba kernel,
76 while array-based analytics operate on its outputs. The weights-and-prices boundary suits
77 portfolio research; order-book events and signal generation remain outside this engine.

78 The public interface is `qis.__all__`, which holds 474 names; `src/qis/api.py` records that
79 list as a literal together with a documented core of 166 symbols grouped by capability, and the
80 suite fails when either record disagrees with the namespace. The broad namespace preserves
81 convenient imports and compatibility for consumers, at the cost of discoverability; the curated
82 core provides a smaller entry point without removing existing exports.

83 Tests check exported plots on synthetic panels, documented core arguments, local
84 documentation links and the measurements quoted here. Selected offline examples execute
85 in the suite; static checks cover example imports and calls, including network-dependent
86 examples. Plot smoke tests establish that figures draw, while layout and readability require
87 visual review.

88 The suite runs without network access on a core installation, from a frozen seeded simulator
89 reproducing the defects of real panels: ragged starts, missing observations, stale prices, appraisal
90 smoothing.

91 Research impact statement

92 The public consumers below are maintained by the author, so they demonstrate research
93 reuse within this stack rather than independent adoption. `optimalportfolios` declares `qis`
94 a mandatory dependency and references 137 of its symbols at 1105 sites. `trendfollowing`,
95 which carries the trend-following work cited below, references 87 symbols at 428 sites. Both
96 counts are taken at the commits recorded in `docs/audit/consumers.json` and reproduced by
97 `tools/audit_consumers.py --pinned`.

98 A third public package, `privateassets`, applies the unsmoothing layer to private-asset returns;
99 we cite it for the range of asset classes served, not as adoption evidence.

100 The capabilities carry named results. The portfolio and optimisation layers support work
101 on robust strategic and tactical allocation (Sepp, Ossa, et al., 2026), the backtester and
102 performance layer work on cryptocurrency allocation (Sepp, 2023), the regime-conditional
103 layer work on trend-following systems (Sepp & Lucic, 2026), and the resampling layer work
104 on capital market assumptions built from multi-asset tradable factors (Sepp, Hansen, et al.,
105 2026). The last two are working papers.

106 Commits run from December 2022 to October 2026, with activity in 40 of the 47 calendar
107 months. External contributions include the numerical, performance and reporting work
108 acknowledged below and recorded in the repository's public history. We do not infer wider
109 research adoption from these contributions.

AI usage disclosure

During July–October 2026, Claude Opus 4.8 and 5.5 and OpenAI Codex assisted with software development, tests, documentation, and paper preparation. The author made the methodological and architectural decisions, reviewed and validated the assisted outputs, and remains responsible for the work.

Acknowledgements

The author thanks John D. Reynolds for contributions to numerical correctness, performance, reporting, and regression test coverage in QIS.

References

- arch contributors. (2026). *arch: StationaryBootstrap*. <https://arch.readthedocs.io/en/stable/bootstrap/generated/arch.bootstrap.StationaryBootstrap.html>
- bt contributors. (2026). *bt: Flexible backtesting for python*. <https://pmorisette.github.io/bt/>
- empyrial-reloaded contributors. (2026). *empyrial-reloaded: Common financial risk and performance metrics*. <https://empyrial.ml4trading.io/>
- ffn contributors. (2026). *ffn: Financial function library for python*. <https://pmorisette.github.io/ffn/>
- Getmansky, M., Lo, A. W., & Makarov, I. (2004). An econometric model of serial correlation and illiquidity in hedge fund returns. *Journal of Financial Economics*, 74(3), 529–609. <https://doi.org/10.1016/j.jfineco.2004.04.001>
- Martin, R. A. (2021). PyPortfolioOpt: Portfolio optimization in python. *Journal of Open Source Software*, 6(61), 3066. <https://doi.org/10.21105/joss.03066>
- Peterson, B. G., & Carl, P. (2026). *PerformanceAnalytics: Econometric tools for performance and risk analysis*. <https://braverock.r-universe.dev/PerformanceAnalytics/doc/manual.html>
- Politis, D. N., & Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303–1313. <https://doi.org/10.1080/01621459.1994.10476870>
- pyfolio-reloaded contributors. (2026). *pyfolio-reloaded: API reference*. <https://pyfolio.ml4trading.io/api-reference.html>
- QuantStats contributors. (2026). *QuantStats: Portfolio analytics for quants*. <https://github.com/ranaroussi/quantstats>
- Riskfolio-Lib contributors. (2026). *Riskfolio-Lib: Portfolio optimization in python*. <https://riskfolio-lib.readthedocs.io/en/latest/>
- Sepp, A. (2023). Optimal allocation to cryptocurrencies in diversified portfolios. *Risk Magazine*, 1–6. <https://www.risk.net/cutting-edge/7957914/optimal-allocation-to-cryptocurrencies-in-diversified-portfolios>
- Sepp, A., & Dezaud, L. (2019). Trend-following CTAs vs alternative risk-premia: Crisis beta vs risk-premia alpha. *The Hedge Fund Journal*, 138, 20–31. <https://thehedgefundjournal.com/trend-following-ctas-vs-alternative-risk-premia/>
- Sepp, A., Hansen, E. H., & Kastenholz, M. (2026). *Capital market assumptions and strategic asset allocation using multi-asset tradable factors* [Working paper]. <https://doi.org/10.2139/ssrn.6785958>

- 151 Sepp, A., & Lucic, V. (2026). *The science and practice of trend-following systems* [Working
152 paper]. <https://doi.org/10.48550/arXiv.2607.19497>
- 153 Sepp, A., Ossa, I., & Kastenholz, M. (2026). Robust optimization of strategic and tactical
154 asset allocation for multi-asset portfolios. *The Journal of Portfolio Management*, 52(4),
155 86–120. <https://doi.org/10.3905/jpm.2025.1.806>
- 156 skfolio contributors. (2026). *skfolio: Portfolio optimization in python*. <https://skfolio.org/>
- 157 vectorbt contributors. (2026). *vectorbt: Portfolio API reference*. [https://vectorbt.dev/api/
158 portfolio/base/](https://vectorbt.dev/api/portfolio/base/)

DRAFT